COMP3028 | SOFTWARE CONSTRUCTION

Data Sources & Node.js

Request payloads $\rightarrow$ Express $\rightarrow$ PostgreSQL

A beginner-friendly, follow-along lecture with runnable examples

How to use these slides

1. **Understand** the idea before writing any code.
2. **Follow** each numbered step in your terminal and editor.
3. **Predict** what the result should look like.
4. **Run and compare** your output with the slide.
5. **Try again independently** at the end of each section.

For students who are new to programming

You do not need prior database experience. Basic JavaScript knowledge is helpful; unfamiliar terms are introduced as we go.

Our journey: from a browser to a database



The big idea

A user *sends data*; Express *receives and validates it*; PostgreSQL *stores and retrieves it*.

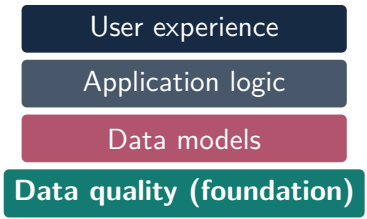
What you will be able to do

Part A: Data sent by clients	Part B: Receiving and storing data
▶ Explain GET, POST, PUT and DELETE.	▶ Read `req.query`, `req.params`, `req.body`.
▶ Recognise query strings, forms and JSON.	▶ Run a small Express server.
▶ Write valid JSON objects and arrays.	▶ Connect to PostgreSQL and run SQL.
	▶ Apply simple database migrations.

Final mini-project: a tiny Book API that accepts and returns database records.

Why data quality matters

- An app is only as useful as the information it works with.
- Missing, duplicated or invalid data leads to wrong results.
- Validate input *before* storing it.
- Use database constraints for an additional safety layer.



Think about it

What happens if someone enters a book title that is an empty string?

Part 1: HTTP request payloads

01

What information travels in an HTTP request?

Goal

Identify *where* information is located in the request before writing Express code to read it.

HTTP means request → response

Request: client to server

- Method: e.g. `GET`, `POST`
- Path: e.g. `/books`
- Headers: metadata about the request
- Optional body: data being sent

Response: server to client

- Status: e.g. `200`, `404`
- Headers: format of the response
- Body: HTML, JSON, text, etc.

Check your progress

Opening a website starts with a request. The website (or its server) sends a response.

A request has several parts

```
POST /api/books HTTP/1.1
Host: localhost:3000
Content-Type: application/json

{"title":"Dune","author":"Frank Herbert"}
```

1. `POST`: HTTP **method** (the action).
2. `/api/books`: **path** (the destination).
3. `Content-Type`: **header** describing the body's format.
4. The JSON after the blank line: **body** (the payload).

The exact headers sent by a real client may include additional fields.

Which HTTP method should I use?

Method	Common purpose	Common place for input
GET	Read a resource	URL query or URL path
POST	Create or submit data	Request body
PUT	Replace or update a resource	Request body
DELETE	Remove a resource	URL path (often an ID)

Important nuance

An HTTP method does not automatically decide whether a body exists. However, a `GET` body has no generally defined semantics and is typically avoided.

Four common ways of sending information

Format	Typical use	Example location
Query string	Filters or searching	`?sort=asc` in URL
URL-encoded	Simple HTML forms	Request body
JSON	Sending structured API data	Request body
Multipart form	Text plus uploaded files	Request body

Remember

`Content-Type` tells the server how a *request body* is encoded; query strings appear in the URL, not in that body.

Query strings: read this URL together

```
http://localhost:3000/books?title=dune&sort=asc
```

- /books: the requested route.
- ?: starts the query string.
- `title=dune`: key `title`, value dune.
- &: separates additional key-value pairs.
- `sort=asc`: key `sort`, value asc.

Try to predict

What are the values of `title` and `sort` if the URL ends in `?title=robot&sort=desc`?

URL-encoded form body

```
POST /feedback HTTP/1.1
Content-Type: application/x-www-form-urlencoded

name=Ana&message=Great+lecture
```

- Form fields become `key=value` pairs.
- The pairs are separated with `&`.
- A `+` often represents a space in this encoding.
- Express can parse this using `express.urlencoded()`.

Difference from the previous slide

Query string: after `?` *in the URL*. URL-encoded form: usually *in the request body*.

JSON request body

```
POST /api/books HTTP/1.1
Content-Type: application/json

{
  "title": "Dune",
  "author": "Frank Herbert"
}
```

- JSON stands for **JavaScript Object Notation**.
- APIs commonly use it to exchange structured information.
- Express can parse this using `express.json()`.

Multipart data: when a user uploads a file

Example form fields

Name: Ana

Profile image: photo.jpg

The request body contains sections separated by a special *boundary*.

What is different?

- It can carry binary file content.
- Plain `express.json()` is not enough.
- We need upload middleware (e.g. `express-fileupload`).

Security reminder

For real systems, restrict file size and type; never trust the uploaded file name alone.

Checkpoint 1: choose the right format

1. A user sorts a list by title. **?**
2. A sign-up form sends name and email. **?**
3. An API sends nested user preferences. **?**
4. A student uploads a PDF assignment. **?**

Discuss before revealing the answer

Likely choices: (1) query string, (2) URL-encoded form or JSON, (3) JSON, (4) multipart form data.

Part 1B: JSON explained slowly

{ }

Objects, arrays, values, and nesting

Why learn JSON?

You will use JSON when an API sends a request and when Express returns a response.

JSON object = named pieces of information

```
{
  "firstName": "Ana",
  "age": 28,
  "isStudent": true
}
```

- Curly braces { } create an **object**.
- Each item is a **key: value** pair.
- Keys must be strings with double quotes.
- Values may be strings, numbers, booleans, `null`, arrays or objects.

JSON array = a list of values

```
{
  "student": "Ana",
  "subjects": ["JavaScript", "SQL", "Testing"],
  "scores": [82, 91, 88]
}
```

- Square brackets `[ ]` create an **array**.
- Items have an order, separated by commas.
- Arrays can contain other arrays or objects.

Check your progress

How many subjects does Ana study? What is the second score?

Nested JSON = data inside data

```
{
  "name": "Ana",
  "address": {
    "city": "Sydney",
    "postcode": "2000"
  },
  "phones": [
    {"type": "home", "number": "123-456"},
    {"type": "mobile", "number": "0400-123"}
  ]
}
```

The value of address is another object. The value of phones is an array of objects.

Common JSON mistakes (and corrections)

Incorrect JSON

```
{
  'name': 'Ana',
  "age": 28,
  "active": True,
}
```

Correct JSON

```
{
  "name": "Ana",
  "age": 28,
  "active": true
}
```

Corrected: double quotes, lowercase `true`, and no trailing comma. JSON is stricter than a JavaScript object literal.

JSON, XML and YAML: same idea, different formats

Format	Main style	Where students may see it
JSON	Braces, arrays, key-value pairs	Web APIs and application data
XML	Nested opening / closing tags	Documents and integrations
YAML	Indentation-based structure	Configuration files and CI pipelines

Today's focus

All three can represent structured data. We will practise JSON because it is especially common in Express APIs.

02

Build a server and inspect incoming requests

What is Express?

Express is a Node.js library that helps us define URL routes and respond to HTTP requests.

What you need before starting

- **Node.js** (an active LTS version) and **npm**.
- A text editor: VS Code is convenient, but not required.
- A terminal: Command Prompt, PowerShell, macOS Terminal, or Bash.
- A browser; optionally an API client such as Postman or VS Code Thunder Client.

Check that Node.js and npm are installed

Run both commands in a terminal. Each should display a version number.

```
node -version      npm -version
```

If a command is not recognised, install Node.js from `https://nodejs.org/` and reopen the terminal.

Step 1: make a project folder

```
mkdir lecture8-data
cd lecture8-data
npm init -y
```

1. `mkdir`: creates a new folder.
2. `cd`: moves into that folder.
3. `npm init -y`: makes `package.json` with defaults.

Check your progress

You should now see `package.json` in the folder. Keep this terminal open.

Step 2: install the libraries

```
npm install express postgres dotenv express-fileupload
```

- `express`: routes and HTTP responses.
- `postgres`: Postgres.js database client.
- `dotenv`: loads local settings from `.env`.
- `express-fileupload`: reads uploaded files.

Expected result

A `node_modules` folder and a `package-lock.json` file appear. You do not need to edit either manually.

Step 3: create the starter files

```
lecture8-data/
  app.js
  package.json
  package-lock.json
  node_modules/
  public/
    form.html
    upload.html
  migrations/
    001_create_books.sql
    002_seed_books.sql
  .env
  .gitignore
```

1. Open your project folder in VS Code.
2. Create the empty folders `public` and `migrations`.
3. Create the files shown at left.
4. You will fill them in as we proceed.

Use the exact file names. File names and paths matter when code refers to them.

Step 4: write your first Express server

File: app.js

```
const express = require('express');
const app = express();

app.get('/', (req, res) => {
  res.send('Hello from Node.js!');
});

app.listen(3000, () => {
  console.log('Server: http://localhost:3000');
});
```

app.get(path, handler) registers a route. res.send() sends an HTTP response.

Step 5: start the server and visit the route

1. Save `app.js`.
2. In the project terminal, type:

```
node app.js
```

3. Open `http://localhost:3000` in a browser.
4. Confirm the page displays `Hello from Node.js!`.

Expected terminal output

```
Server: http://localhost:3000
```

To stop the server, press Ctrl+C in the terminal. Restart with `node app.js` after code changes.

The three parts of an Express route

```
app.get('/', (req, res) => { ... });
```

`get`	`'/'`	`(req,res)`
HTTP method	URL path	Handler inputs

Two objects to remember

`req` contains information *received* from the client. `res` lets the server *send back* a response.

Step 6: add middleware *before* your routes

Add near the top of `app.js`, after `const app = express();`

```
app.use(express.json());
app.use(express.urlencoded({ extended: false }));
```

- `express.json()` parses JSON request bodies.
- `express.urlencoded()` parses URL-encoded form bodies.
- `extended: false` is adequate for simple form fields.

Ordering matters

Middleware runs in registration order. Add these lines *before* routes that need `req.body`.

Step 7: read a query string with `req.query`

Add this route above `app.listen(...)`.

```
app.get('/hello', (req, res) => {
  const name = req.query.name || 'student';
  res.json({ message: `Hello, ${name}!` });
});
```

1. Restart the server.
2. Visit `http://localhost:3000/hello?name=Ana`.
3. Change `Ana` to your own name and reload.

Check your progress

Expected JSON: `{"message":"Hello, Ana!"}`

Step 8: read a URL parameter with `req.params`

Add this route above `app.listen(...)`.

```
app.get('/product/:id', (req, res) => {
  res.json({ productId: req.params.id });
});
```

- `:id` means “capture this part of the URL”.
- Visit `http://localhost:3000/product/42`.
- Response: `{"productId":"42"}`.
- URL parameters are strings until you convert them.

Do not confuse these three objects

Express property	Example request	Result
`req.query`	`/hello?name=Ana`	`req.query.name` is `"Ana"`
`req.params`	`/product/42` (route `/product/:id`)	`req.params.id` is `"42"`
`req.body`	POST with JSON or form fields	`req.body.name` from parsed body

Memory trick

Query = after ?; **params** = named path part; **body** = data inside a request.

Step 9A: create a basic HTML form

File: `public/form.html`

```
<!doctype html>
<html lang="en">
  <head><meta charset="utf-8"><title>Feedback</title></head>
  <body>
    <h1>Send feedback</h1>
    <form action="/feedback" method="POST">
      <label>Name <input name="name" required></label>
      <label>Message <input name="message" required></label>
      <button type="submit">Send</button>
    </form>
  </body>
</html>
```

Why these attributes matter

`action` is the destination; `method` is POST; `name` labels each form field.

Step 9B: serve the form and receive its data

In app.js, add path near the top:

```
const path = require('node:path');
```

Then add these routes above app.listen(...):

```
app.get('/form', (req, res) => {
  res.sendFile(path.join(__dirname, 'public', 'form.html'));
});

app.post('/feedback', (req, res) => {
  const { name, message } = req.body;
  res.json({ receivedFrom: name, feedback: message });
});
```

Step 9C: try the form in a browser

1. Restart the Node.js server.
2. Open `http://localhost:3000/form`.
3. Type your name and a short message.
4. Click **Send**.
5. Inspect the JSON response in the browser.

Expected response for Ana and “Useful lecture”

```
{"receivedFrom":"Ana","feedback":"Useful lecture"}
```

Submitting the form sends a POST request with URL-encoded data.

Step 10A: create a JSON receiver

Add this route in `app.js`:

```
app.post('/api/echo', (req, res) => {
  const name = req.body.name;
  if (typeof name !== 'string' || !name.trim()) {
    return res.status(400).json({ error: 'Name required' });
  }
  res.json({ received: name });
});
```

- It expects a JSON object with a `name` field.
- `res.status(400)` means invalid request data.
- `return` prevents the handler sending a second response.

Step 10B: send JSON to the server

Using Postman / Thunder Client:

1. Create a new HTTP request and select `POST`.
2. URL: `http://localhost:3000/api/echo`
3. Choose **Body** → **raw JSON**.
4. Enter the following and click **Send**:

```
{
  "name": "Ana"
}
```

Check your progress

Expected HTTP 200 response: `{"received":"Ana"}`. Try `{}` next: what status do you get?

Step 11A: create an upload form

File: `public/upload.html`

```
<!doctype html>
<html lang="en">
  <head><meta charset="utf-8"><title>Upload</title></head>
  <body>
    <h1>Try a file upload</h1>
    <form action="/upload" method="POST"
          enctype="multipart/form-data">
      <input type="file" name="file" required>
      <button type="submit">Upload</button>
    </form>
  </body>
</html>
```

The enctype value is essential for uploading files from an HTML form.

Step 11B: enable multipart middleware

In `app.js`, near the top:

```
const fileUpload = require('express-fileupload');
```

Add before your routes, below the other middleware:

```
app.use(fileUpload({
  limits: { fileSize: 2 * 1024 * 1024 },
  abortOnLimit: true
}));
```

- The size limit is 2 MiB for this classroom example.
- Uploaded files become available through `req.files`.

Step 11C: receive an uploaded file

Add these routes above `app.listen(...)`:

```
app.get('/upload-form', (req, res) => {
  res.sendFile(path.join(__dirname, 'public', 'upload.html'));
});

app.post('/upload', (req, res) => {
  const file = req.files?.file;
  if (!file) {
    return res.status(400).json({ error: 'No file sent' });
  }
  res.json({ filename: file.name, bytes: file.size });
});
```

Test

Restart the server; open `/upload-form`; select a small file. This demo returns only file metadata. It does *not* save the upload.

Checkpoint 2: can you explain each result?

1. `GET /hello?name=Ada` → which request property?
2. `GET /product/7` → which request property?
3. `POST /feedback` → which request property?
4. `POST /api/echo` (JSON) → which middleware?
5. `POST /upload` → which middleware?

Answers

(1) `req.query`; (2) `req.params`; (3) `req.body`; (4) `express.json()`; (5) `express-fileupload`.

03

Store data that survives a server restart

Where we are going

Create a books table, fill it with sample rows, and let Express read and add books.

What is a database?

- A database stores information beyond a single web request.
- A **table** has columns (fields) and rows (records).
- **SQL** is a language for asking questions and changing data.
- PostgreSQL ("Postgres") is a popular open-source relational database.

id	title	author
1	Dune	Frank Herbert
2	The Hobbit	Tolkien

One row = one book. Each column = one piece of information.

Why PostgreSQL? Key capabilities in plain language

- **ACID transactions**: changes are handled reliably.
- **Concurrency / MVCC**: many users can work with data at once.
- **SQL + JSON support**: relational tables and JSON features.
- **Indexes**: speed up some searches on large datasets.
- **Replication**: help copy data to other database servers.
- **Constraints**: reject invalid records, such as a missing title.

Later database courses will explore these mechanisms in more depth. Today we focus on hands-on SQL and a Node.js connection.

Step 12: choose a PostgreSQL environment

Option A: existing course Dev Container

If your container already runs PostgreSQL, use its supplied host, port, database, username and password. No new Docker database is required.

Option B: Docker on your own computer

If Docker is installed, follow the next slide to start a throwaway local database.

Option C: local installation or hosted database

PostgreSQL: `https://www.postgresql.org/download/`
Hosted example: `https://neon.tech/`

Choose *one* setup. In a remote or hosted setup, your connection values will differ.

Step 13: start a local PostgreSQL with Docker

Only if you selected Option B:

```
docker run --name lecture8-db \
  -e POSTGRES_PASSWORD=postgres \
  -e POSTGRES_DB=lecture8 \
  -p 5432:5432 -d postgres:16
```

- Docker downloads PostgreSQL 16 if necessary.
- `lecture8` is the database name.
- `postgres` is the *demo-only* password.
- `5432` is the default PostgreSQL port.

Windows PowerShell / Command Prompt

Type the command on *one line*, omitting the backslashes above (which are Bash line-continuation characters).

Step 14: check the database is running

```
docker ps
```

- Look for a container named `lecture8-db`.
- You should see port mapping `0.0.0.0:5432->5432/tcp` or a platform equivalent.
- If it was stopped, run `docker start lecture8-db`.

Optional: open PostgreSQL's command-line tool

```
docker exec -it lecture8-db psql -U postgres -d lecture8
```

Once inside, type `\q` to exit.

Before SQL: understand our Book table

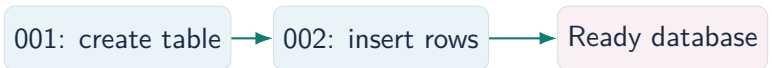
Column	SQL type	Purpose
id	SERIAL	An automatically assigned number
title	TEXT	The book title; mandatory
author	TEXT	The author's name; mandatory
created_at	TIMESTAMPTZ	When the row was created

Primary key

`id` uniquely identifies each row. Two different books should not share the same `id`.

What is a migration?

- A **migration** is a versioned file that changes the database's structure or data.
- Teams store migration files alongside their code.
- Applying files in order lets team members build a consistent database.



A real project typically uses a migration runner to track which migrations have already been applied. Here we apply two files manually for clarity.

Step 15: write migration 001

File: `migrations/001_create_books.sql`

```sql
CREATE TABLE IF NOT EXISTS books (
  id SERIAL PRIMARY KEY,
  title TEXT NOT NULL,
  author TEXT NOT NULL,
  created_at TIMESTAMPTZ DEFAULT NOW()
);
```

- `CREATE TABLE` makes the table.
- `NOT NULL` means a value must be provided.
- `DEFAULT NOW()` records the creation timestamp.

Step 16: write migration 002

File: `migrations/002_seed_books.sql`

```
INSERT INTO books (title, author) VALUES
  ('Dune', 'Frank Herbert'),
  ('The Hobbit', 'J. R. R. Tolkien');
```

- `INSERT INTO` creates new records.
- The column names must match the table.
- `VALUES` supplies data for each row.

Note

Running this seed file again adds more rows. Apply it only once in this beginner exercise unless you intentionally reset the database.

Step 17: apply migration 001

For the Docker option, from the project folder:

```
docker cp migrations/001_create_books.sql \
  lecture8-db:/tmp/001.sql

docker exec lecture8-db psql -U postgres -d lecture8 \
  -f /tmp/001.sql
```

Expected result

```
CREATE TABLE
```

On Windows, type each long command on one line rather than using a trailing backslash. If using a course Dev Container, run `psql` with your instructor's connection details instead.

Step 18: apply migration 002

For the Docker option, from the project folder:

```
docker cp migrations/002_seed_books.sql \
  lecture8-db:/tmp/002.sql

docker exec lecture8-db psql -U postgres -d lecture8 \
  -f /tmp/002.sql
```

Expected result

`INSERT 0 2` means *two rows inserted*.

Step 19: inspect the data with SQL

Open psql (Docker option):

```
docker exec -it lecture8-db psql -U postgres -d lecture8
```

Inside psql, type:

```
SELECT id, title, author FROM books ORDER BY id;
```

Expected rows

```
1 | Dune | Frank Herbert
2 | The Hobbit | J. R. R. Tolkien
```

After checking, type \q to exit psql. Generated IDs may differ if you have previously inserted rows.

SQL essentials: what these words do

SQL word	Meaning
`SELECT`	Which columns should be returned?
`FROM`	Which table should be read?
`WHERE`	Which rows match a condition?
`ORDER BY`	In what order should rows be returned?
`INSERT INTO`	Which table should receive a new row?
`RETURNING`	Which inserted values should be sent back?

Check your progress

Translate `SELECT title FROM books;` into plain English.

Step 20: configure Node.js database access

File: `.env` (Docker setup shown)

```
DATABASE_URL=postgres://postgres:postgres@localhost:5432/lecture8
PORT=3000
```

File: `.gitignore`

```
node_modules/
.env
```

- `DATABASE_URL` has username, password, host, port and database.
- `.gitignore` helps avoid accidentally committing private settings.

Credentials

The password above is only a deliberately simple *local class demonstration*. Never reuse it for a public database.

Step 21: add a Postgres.js connection

At the top of `app.js`, add:

```
require('dotenv').config();
const postgres = require('postgres');
const sql = postgres(process.env.DATABASE_URL);
```

- dotenv reads the `.env` file.
- `postgres(...)` creates a client for future SQL queries.
- A query can use `await` within an async function.

Common beginner mistake

Keep `require('dotenv').config()` *before* accessing `process.env.DATABASE_URL`.

Step 22: test the connection through a route

Add this Express route to `app.js`:

```
app.get('/db-check', async (req, res) => {
  try {
    const rows = await sql`SELECT NOW() AS server_time`;
    res.json({ databaseTime: rows[0].server_time });
  } catch (err) {
    console.error(err);
    res.status(500).json({ error: 'Database unavailable' });
  }
});
```

1. Restart Node.js; visit `http://localhost:3000/db-check`.
2. Expected: JSON with a database timestamp.
3. If it fails, check the Docker container and `.env`.

How Postgres.js sends SQL

Important new syntax: a *tagged template literal*

Instead of passing a normal string, Postgres.js uses backticks after `sql`.

```
const rows = await sql`SELECT * FROM books`;
```

- `await` waits for the database result.
- `rows` is an array of JavaScript row objects.
- We use this result to construct an HTTP response.

Step 23: read books from PostgreSQL

Add this route to `app.js`:

```
app.get('/api/books', async (req, res) => {
  try {
    const rows = await sql`
      SELECT id, title, author FROM books ORDER BY id
    `;
    res.json(rows);
  } catch (err) {
    console.error(err);
    res.status(500).json({ error: 'Database error' });
  }
});
```

Test

Restart and open `http://localhost:3000/api/books`. Expect a JSON *array* containing the two seeded books.

What does the database result look like?

```
[
  {"id":1,"title":"Dune","author":"Frank Herbert"},
  {"id":2,"title":"The Hobbit",
   "author":"J. R. R. Tolkien"}
]
```

- Each table row becomes a JavaScript object.
- Many row objects form an array.
- `res.json(rows)` serialises the array into JSON.

Your output may have different spacing or IDs, depending on your database state.

Step 24: insert a book from an HTTP POST

Add this route above `app.listen(...)`:

```
app.post('/api/books', async (req, res) => {
  const { title, author } = req.body;
  if (typeof title !== 'string' || !title.trim() ||
      typeof author !== 'string' || !author.trim()) {
    return res.status(400).json({ error: 'Title and author required' });
  }
  try {
    const rows = await sql`
      INSERT INTO books (title, author)
      VALUES (${title.trim()}, ${author.trim()})
      RETURNING id, title, author
    `;
    res.status(201).json(rows[0]);
  } catch (err) {
    console.error(err);
    res.status(500).json({ error: 'Database error' });
  }
});
```

Why `${...}` are safer here

- A Postgres.js **tagged template** separates SQL structure from values.
- In `VALUES (${title}, ${author})`, those inputs are safely bound as query parameters.
- This helps prevent **SQL injection**.
- It is *not* permission to skip application input checks.

Never do this

Do not build SQL by joining untrusted user input into a SQL string. Keep data in bound values.

This protection concerns SQL injection; it is distinct from cross-site scripting (XSS).

Step 25: test the insert with an API client

1. Open Postman / Thunder Client and select `POST`.
2. URL: `http://localhost:3000/api/books`.
3. Set the body to **raw JSON**; enter:

```
{
  "title": "Clean Code",
  "author": "Robert C. Martin"
}
```

4. Click **Send**. Expect HTTP **201 Created** with the new book ID.
5. Open `GET /api/books` and confirm the book was added.

Step 26: fetch exactly one book by ID

Add to app.js:

```
app.get('/api/books/:id', async (req, res) => {
  const id = Number(req.params.id);
  if (!Number.isSafeInteger(id) || id <= 0) {
    return res.status(400).json({ error: 'Invalid ID' });
  }
  try {
    const rows = await sql`
      SELECT id, title, author FROM books WHERE id = ${id}
    `;
    if (!rows.length) {
      return res.status(404).json({ error: 'Book not found' });
    }
    res.json(rows[0]);
  } catch (err) {
    console.error(err);
    res.status(500).json({ error: 'Database error' });
  }
});
```

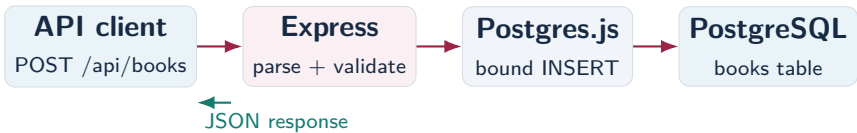
Step 27: test success and error cases

Request	Expected status / meaning
`GET /api/books`	200 – array of books
`GET /api/books/1`	200 – first book (if present)
`GET /api/books/abc`	400 – malformed ID
`GET /api/books/999999`	404 – book does not exist
`POST /api/books` with no author	400 – invalid request body
`POST /api/books` with valid JSON	201 – created a new book

Check your progress

Why is `404` different from `400`? Explain in one sentence.

Putting everything together



1. The client sends **JSON** in a POST request.
2. Express **parses and checks** the fields.
3. Postgres.js **binds parameters** and executes SQL.
4. PostgreSQL **stores** the new row and returns its ID.

Migration workflow for teams

1. Developer A adds `003_add_isbn.sql`.
2. They commit the new migration file with their application changes.
3. Developer B pulls the latest repository state.
4. Developer B applies migration 003 to their own database.
5. Both databases now contain the new column.

What the lecture's diagram is illustrating

Database evolution is a *sequence of changes*, not a file copied over somebody else's database. Production projects use migration tools to track applied versions and manage failures.

Optional exercise: create a third migration

File: `migrations/003_add_isbn.sql`

```
ALTER TABLE books
ADD COLUMN isbn TEXT;
```

- Apply it using the same Docker copy and `psql -f` steps.
- Use `\d books` inside `psql` to inspect the table structure.
- Explain why a nullable new column leaves old book rows valid.

Extension task only: the API routes above do not yet read or write `isbn`.

Troubleshooting: start with the error message

What you see	First thing to check
`node: command not found`	Install Node.js and reopen terminal
`Cannot find module 'express'`	Run `npm install` in project folder
Browser cannot connect	Is `node app.js` still running?
`req.body` is undefined	Parser middleware and `Content-Type`
Database connection refused	PostgreSQL running? Correct host/port?
`relation "books" does not exist`	Did migration 001 run in correct database?

Debugging routine students can reuse

1. **Read** the complete message: terminal and HTTP response.
2. **Locate** the failing layer: browser, Express or PostgreSQL.
3. **Simplify**: test `/`, then `/db-check`, then `/api/books`.
4. **Check** file names, port, JSON syntax and database credentials.
5. **Change one thing**, restart and test again.

A useful habit

If `GET /` succeeds but `GET /db-check` fails, Express probably works; focus on the database connection instead.

Practice task 1: small changes, big learning

1. Change `/hello` to display `Welcome, name!`.
2. Add `?course=COMP3028` and return the course in JSON.
3. Create `GET /square/:number` that returns the square of a valid number.
4. Send `{"name":"Kai"}` to `POST /api/echo`.

Evidence to show your tutor

The request URL / body, the resulting response, and a short explanation of `req.query`, `req.params` or `req.body` used.

Practice task 2: change the Book API

1. Add one more book through `POST /api/books`.
2. Retrieve that book using its returned ID.
3. Send an empty title and observe the error.
4. Explain which line prevents a missing title from being inserted.
5. **Challenge:** create `GET /api/books/search?author=...` with a parameterised SQL query.

Do not forget

Put specific routes such as `/api/books/search` *before* the generic `/api/books/:id` route, otherwise `search` may be interpreted as an ID.

Self-check: can you answer without notes?

1. How is a query string different from a request body?
2. Why does Express need JSON parsing middleware?
3. How do you read `:id` from `/product/:id`?
4. What is one reason to use PostgreSQL rather than an in-memory array?
5. What do migration files help teams coordinate?
6. Why should user input be a *bound SQL parameter*?

Success criterion

You can explain the six answers *and* demonstrate at least one successful database-backed API request.

Key takeaways

1. **Payloads** carry values through URLs and request bodies.
2. **JSON** organises data using objects and arrays.
3. **Express** reads inputs from `req.query`, `req.params`, and `req.body`.
4. **Middleware** prepares form, JSON and file-upload data.
5. **PostgreSQL** persists data in tables; **Postgres.js** connects Node.js to SQL.
6. **Migrations** make database changes reproducible.

One sentence to remember

Client request → Express route → validated input → SQL query → JSON response.

References and further learning

- **Source lecture:** COMP3028 Software Construction, *Lecture 8: Data Sources & Node.js*, supplied course slides (51 pages). This teaching edition preserves its three main topics and adds independently runnable examples.
- Node.js getting started: https://nodejs.org/en/learn
- Express guide: https://expressjs.com/en/guide/routing.html
- Express API: https://expressjs.com/en/api.html
- Postgres.js documentation: https://github.com/porsager/postgres
- PostgreSQL tutorial: https://www.postgresql.org/docs/current/tutorial.html
- PostgreSQL downloads: https://www.postgresql.org/download/

You built a data-backed API!

Keep experimenting: change one input, predict, run, explain.

Next step: add more routes and tests to your Book API.